

Bare VPS to live system

Ten stages, in order, from an untouched Ubuntu box to a running, monitored SP-MIS. Three scripts do the mechanical work; the rest is decisions and verification that should not be automated.

BEFORE YOU START

You need: an **Ubuntu 22.04/24.04 VPS with at least 8 GB RAM**, a domain with an **A** record pointing at it, ports 80/443 open, and SSH key access as root.

8 GB is not padding. `docker-compose.prod.yml` declares exactly **4,096 MB** of container memory limits. On a 4 GB box the host has nothing left and the kernel starts killing containers — usually Postgres, usually at night. `provision.sh` checks this before it changes anything.

The three scripts

SCRIPT	WHEN	WHAT IT DOES
<code>scripts/provision.sh</code>	ONCE	Hardens the host: user, SSH, firewall, fail2ban, Docker, swap, log rotation
<code>scripts/deploy.sh</code>	EVERY RELEASE	Backs up, switches the tag, pulls, starts, waits for health, verifies
<code>scripts/verify.sh</code>	AFTER EVERY DEPLOY	Proves the stack works — including the failures that are silent

All three are shellcheck-clean and idempotent. None has been run against a real server yet — this deployment is their first execution.

Harden the host

```
scp scripts/provision.sh root@<server-ip>:/root/
ssh root@<server-ip>
chmod +x provision.sh

# ALWAYS audit first. Changes nothing; prints what is already done.
./provision.sh --check

DEPLOY_USER=deploy ./provision.sh
```

Creates the `deploy` user, merges your root SSH key to it, disables root login and password auth, enables UFW (SSH rate-limited, 80/443 open), installs fail2ban and Docker, adds 2 GB swap, caps container logs, sets the timezone, and creates `/opt/spmis`.

On a host you configured by hand, run `--check` first. The script skips work already done and compares config files before rewriting them, so it will not clobber your manual `sshd` or `fail2ban` settings, and it restarts a service only when something actually changed. `--check` lets you see that before trusting it. Add `SKIP_UPGRADE=yes` if you have already run `apt-get upgrade`.

It refuses to run if the `deploy` user has no SSH key. That is deliberate: disabling password auth and root login with no key is an unrecoverable lockout. If it stops here, run `ssh-copy-id deploy@<ip>` from your machine and re-run.

If it reports a reboot is required, reboot *before* stage 2 — so you verify SSH on the kernel you will actually run.

02

Prove you are not locked out GATE

```
# Keep the root session OPEN while you test this.
ssh deploy@<server-ip>      # must SUCCEED
ssh root@<server-ip>        # must be REFUSED
```

Only close the root session once both behave as above. Then log back in as `deploy` and confirm Docker works without `sudo` — the group change needs a fresh login:

```
docker run --rm hello-world
```

Take provider snapshot #1 now, labelled `baseline-hardened-docker`.

03

Authenticate to GHCR

Create a classic PAT at github.com/settings/tokens with the single scope `read:packages`. Full walkthrough in [docs/GHCR-Setup-Runbook.pdf](#).

```
ssh deploy@169.58.143.63

echo "ghp_XXXXXXXXXXXX" | docker login ghcr.io -u naynie41 --password-stdin
chmod 600 ~/.docker/config.json

# Prove it before going further — this fails loudly if the token,
# the tag, or the package visibility is wrong.
docker manifest inspect ghcr.io/naynie41/spmis-api:v.1.0.0 > /dev/null && echo API OK
docker manifest inspect ghcr.io/naynie41/spmis-web:v.1.0.0 > /dev/null && echo WEB OK
```

Log in as `deploy`, not `zqnmusa`. Docker credentials live in the invoking user's `~/.docker/config.json`. `deploy` runs the stack, so `deploy` must hold the login — and never pull under `sudo`, which reads root's config instead.

Your release tag is `v.1.0.0` — with a dot after the `v`. The workflow tags images from the git tag verbatim, so the image is `spmis-api:v.1.0.0`. Every example that says `v1.0.0` will fail to pull. Use the tag you actually pushed, or push a clean `v1.0.0` tag and let the workflow rebuild.

Get the deploy files

```
cd /opt/spmis  
git clone --branch v.1.0.0 --depth 1 https://github.com/naynie41/JSG-SP-MIS.git .
```

Only `docker-compose.prod.yml`, `docker/nginx/prod.conf.template` and `scripts/` are used from the checkout. All application code arrives as a pre-built image — the VPS never builds.

`/opt/spmis` already exists and is owned by `deploy`, with `acme/` in place. Clone *into* it (note the trailing `.`); do not recreate it.

Create the environment file

```
cd /opt/spmis
cp .env.prod.example .env

# Generate the two keys from the image itself – no PHP on the host.
# --entrypoint php is REQUIRED (see below).
docker run --rm --entrypoint php ghcr.io/naynie41/spmis-api:v.1.0.0 artisan
key:generate --show
docker run --rm --entrypoint php ghcr.io/naynie41/spmis-api:v.1.0.0 artisan
backup:keygen

nano .env
chmod 600 .env          # owned by deploy – see the warning below
```

Without `--entrypoint php` both commands fail with **FATAL: APP_KEY is not set – refusing to start**. The image entrypoint guards on `APP_KEY`, so the container will not boot to run the very command that generates it. Bypassing the entrypoint runs artisan directly.

What to set. Everything else in the file has a working default.

KEY	VALUE
GHCR_OWNER	naynie41 — lowercase, always
IMAGE_TAG	v.1.0.0 — the dot is real
APP_KEY	from <code>key:generate --show</code>
BACKUP_ENCRYPTION_KEY	from <code>backup:keygen</code>
DB_PASSWORD + POSTGRES_PASSWORD	same strong value in both
REDIS_PASSWORD, RABBITMQ_PASSWORD	strong, distinct
MAIL_*	SMTP host, user, password
AWS_*, AWS_BUCKET	offsite backup store

Your domain appears in seven places. Change every one:

```
APP_URL=https://<domain>
SPA_URL=https://<domain>
CORS_ALLOWED_ORIGINS=https://<domain>
SANCTUM_STATEFUL_DOMAINS=<domain>          # no scheme
MAIL_FROM_ADDRESS="no-reply@<domain>"
NGINX_SERVER_NAME=<domain>
NGINX_SSL_CERTIFICATE=/etc/letsencrypt/live/<domain>/fullchain.pem
NGINX_SSL_CERTIFICATE_KEY=/etc/letsencrypt/live/<domain>/privkey.pem
```

Miss `CORS_ALLOWED_ORIGINS` or `SANCTUM_STATEFUL_DOMAINS` and the site loads but every login fails — the SPA is refused by its own API. Note Sanctum takes the bare host, with no `https://`.

Do not `chown root:root .env`. Compose reads `.env` as the user invoking it, and every command here runs as `deploy` without `sudo`. Root-owned mode 600 makes it unreadable and every compose command fails. `deploy`-owned mode 600 is the correct control — no other unprivileged account can read it.

APP_KEY is generated once and never rotated with data present. It decrypts NIN, BVN and TOTP secrets. Changing it orphans every encrypted column and every backup taken before the change. Store `BACKUP_ENCRYPTION_KEY` *separately* from the `AWS_*` credentials — one breach must not yield both the backups and the key that opens them.

Obtain the TLS certificate

DNS first. Let's Encrypt proves control by fetching a file over HTTP, so the domain must already resolve to this server:

```
dig +short <domain>          # must print 169.58.143.63
```

Certbot is not installed yet:

```
sudo apt-get install -y certbot

# Port 80 must be free — nginx is not running yet, so standalone is correct.
sudo certbot certonly --standalone -d <domain> --agree-tos -m you@example.com

sudo ls /etc/letsencrypt/live/<domain>/      # fullchain.pem + privkey.pem
```

This must happen before the first `up`, not after. nginx reads `ssl_certificate` at startup; if the file is not there the container fails to start, and because `nginx` is the only service publishing 80/443, nothing is reachable. Order matters here.

Now switch renewal off standalone, immediately. Certbot records the method it used in `/etc/letsencrypt/renewal/<domain>.conf` and enables `certbot.timer`. Standalone binds port 80 itself — but from the moment nginx starts, nginx owns port 80. Every future renewal then fails silently on a timer, and the first symptom is the entire site going down on an expired certificate about 90 days later.

```
sudo nano /etc/letsencrypt/renewal/<domain>.conf
```

```
authenticator = webroot
webroot_path = /opt/spmis/acme,
renew_hook = cd /opt/spmis && docker compose -f docker-compose.prod.yml exec -T nginx
nginx -s reload

[[webroot_map]]
<domain> = /opt/spmis/acme
```

nginx already serves that directory at `/.well-known/acme-challenge/`, so webroot renewal works with the stack up. `verify.sh` fails if the authenticator is still `standalone`. Run `sudo certbot renew --dry-run` to confirm — but only *after* stage 7, since nginx must be up to serve the challenge.

THE VPS • AS DEPLOY

07

First deploy

```
cd /opt/spmis
./scripts/deploy.sh v1.0.0
```

The script checks the tag is pullable *before* touching anything, then pulls, starts, and waits for every service to report healthy.

On a first deploy the pre-deploy backup step has nothing to back up and may fail. Use `./scripts/deploy.sh v1.0.0 --no-backup` for this one run only.

The `api` container runs `migrate --force` automatically on boot. The worker and scheduler deliberately do not, so three containers cannot race the same migration.

Install the operating configuration CRITICAL

These are *configuration*, not sample data. Run each by class:

```
C="docker compose -f docker-compose.prod.yml exec api php artisan"

$C db:seed --class=RolesAndPermissionsSeeder --force # roles + permissions
$C db:seed --class=MatchingConfigSeeder --force # SEE BELOW
$C db:seed --class=DoubleDippingRuleSeeder --force
$C db:seed --class=ReferralSlaSeeder --force
$C db:seed --class=GrievanceSlaSeeder --force

# Ward reference data (162 of ~287 wards ship; 6 LGAs have none).
$C reference:load-divisions

# The first administrator. Password is PROMPTED, never an argument.
$C spmis:create-admin admin@your-domain --name="SP-MIS Administrator"
```

Without **MatchingConfigSeeder** there is no active matching config, and the import pipeline skips duplicate screening entirely. No error is raised. No warning is shown. The registry simply fills with duplicates — the one thing this system exists to prevent. **verify.sh** checks for this explicitly in the next stage.

Never run **db:seed --force** on its own: **DatabaseSeeder** also lists the sample-data seeders. Each refuses to run in production, but naming the ones you want makes that a decision rather than a reliance on someone else's guard.

Verify

```
./scripts/verify.sh
```

Checks containers, that only nginx publishes ports, HTTP→HTTPS, API health, HSTS, certificate expiry, pending migrations, **active matching config**, roles, an administrator, ward data, and the audit chain.

Then open the domain and sign in. MFA enrolment is forced at first login.

Take provider snapshot #2, labelled **app-live-preconfig**.

Monitoring — the three essentials

Full detail in `docs/DEPLOY.md` §8. Monitoring lives *outside* the app: a dashboard rendering "all healthy" from inside a broken stack is worse than none.

Uptime

```
docker run -d --restart unless-stopped \
  -p 127.0.0.1:3001:3001 -v uptime-kuma:/app/data \
  --name uptime-kuma louislam/uptime-kuma:1

# Reach it over an SSH tunnel, never publicly:
ssh -L 3001:127.0.0.1:3001 deploy@<server-ip>
```

Monitor `/healthz`, `/api/v1/health`, and TLS expiry (alert at 21 days).

Application errors

Set `SENTRY_LARAVEL_DSN` and `VITE_SENTRY_DSN`. Enable data scrubbing and set `send_default_pii=false` — an exception payload can carry a request body, and a request body here can carry a NIN.

Backup success — a dead man's switch

```
docker compose -f docker-compose.prod.yml exec -T scheduler php artisan backup:run \
  && curl -fsS --retry 3 "$BACKUP_HEARTBEAT_URL" \
  || curl -fsS --retry 3 "$BACKUP_HEARTBEAT_URL/fail"
```

This is the one that matters most. A backup that stops running is silent — nothing fails, nothing alerts, and you find out when you need a restore. Invert it: the backup pings on success, and the service alerts when the ping does not arrive.

TLS renewal

```
sudo certbot renew --webroot -w /opt/spmis/acme \
  --deploy-hook "docker compose -f /opt/spmis/docker-compose.prod.yml exec nginx nginx
  -s reload"
```

Add to cron or a systemd timer — twice daily is standard.

Ongoing: release and rollback

```
# New release — backs up first, then pulls and verifies.
cd /opt/spmis
git fetch --tags && git checkout v1.1.0      # refresh compose/nginx config
./scripts/deploy.sh v1.1.0

# Rollback to the previous release.
./scripts/deploy.sh v1.0.0 --rollback
```

Rolling back code does not roll back the database. Migrations are forward-only. If the release you are leaving ran a destructive migration, restore the database from the backup taken before *it* was deployed. That is why `deploy.sh` backs up before every release, and why it records the tag you came from in `.last-deployed-tag`.

Before real beneficiary data

This system goes live with real beneficiary data. The penetration test, DPO sign-off and retention gates were **lifted by the system owner on 31 August 2026** and are recorded as accepted open risk in `docs/DEPLOY.md` §9. Lifting them changed no control in the system.

One gate was not lifted, and stage 5 is your last chance at it: OPS-02, encryption at rest for the database volume. Every other open item can be closed later — a pen test can be commissioned next quarter, retention periods can be switched on without a redeploy. This one cannot. A volume that was not encrypted before the first real import cannot be retrospectively encrypted for the data already written to it; closing it later means migrating the database onto a new encrypted volume. Confirm it before you import anyone.

Retention staying off is fail-safe, not fragile: nothing is deleted, anonymised, or aged out. The exposure runs the other way — personal data accumulates indefinitely. Both switches are runtime configuration, so the DPO can enable retention later with a restart and no migration.

Commands verified against `docs/DEPLOY.md`, `docker-compose.prod.yml` and the running application. Companion documents: `docs/GHCR-Setup-Runbook.pdf` (stage 3) and `docs/DEPLOY.md` §8–§9 (monitoring and readiness).